

МИНОБРНАУКИ РОССИИ

РГУ НЕФТИ И ГАЗА (НИУ) ИМЕНИ И.М. ГУБКИНА

Факультет Автоматики и вычислительной техники

Кафедра Прикладной математики и компьютерного моделирования

Оценка комиссии: _____ Рейтинг: _____

Подписи членов комиссии:

_____	Прядко С.А.
(подпись)	(фамилия, имя, отчество)
_____	Фомочкина А.С.
(подпись)	(фамилия, имя, отчество)

(дата)	

КУРСОВАЯ РАБОТА

по дисциплине Графика и интерфейсы компьютерных систем

на тему Создание интерфейса для моделирования процесса

«К ЗАЩИТЕ»

Доцент, к.т.н. Прядко С.А.

(должность, ученая степень; фамилия, и.о.)

(подпись)

(дата)

ВЫПОЛНИЛ:

Студент группы АМ-24-09

(номер группы)

Аксёнов Дмитрий Игоревич

(фамилия, имя, отчество)

(подпись)

(дата)

МИНОБРНАУКИ РОССИИ

РГУ НЕФТИ И ГАЗА (НИУ) ИМЕНИ И.М.
ГУБКИНА

Факультет автоматики и вычислительной техники
Кафедра прикладной математики и компьютерного моделирования

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

по дисциплине Графика и интерфейсы компьютерных систем

на тему Создание интерфейса для моделирования процесса

ДАНО студенту Аксёнову Дмитрию Игоревичу группы АМ-24-09
(фамилия, имя, отчество в дательном падеже) (номер группы)

Содержание работы:

1. Введение.
2. Цель и задачи работы, выбор процесса для моделирования.
3. Построение модели процесса.
4. Разработка интерфейса и создание программы
5. Заключение.
6. Список используемой литературы.

Исходные данные для выполнения работы:

1. Источники информации, которые определены на индивидуальной консультации при обсуждении тематики.

Рекомендуемая литература:

1. Прохоренок Н. А. Python 3 и PyQt 6. Разработка приложений. / Н.А. Прохоренок, В.А. Дронов. - Санкт-Петербург : БХВ-Петербург, 2023. - 832 с.
2. Шелудько, В. М. Язык программирования высокого уровня Python. Функции, структуры данных, дополнительные модули: учебное пособие / В. М. Шелудько. – Ростов-на-Дону, Таганрог: Издательство Южного федерального университета, 2017. – 107 с.
3. Папилина Т.М., Прядко С.А., Фомочкина А.С. Применение модулей языка PYTHON в инженерных расчетах / Папилина Т.М., Прядко С.А., Фомочкина А.С. — М. РГУ нефти и газа (НИУ) имени И.М. Губкина, 2021. — 141 с.

Графическая часть:

1. отсутствует

Требования к представлению результатов:

✓	Электронная версия
	Бумажный вариант и электронный образ документа

Руководитель: к.т.н. доцент Прядко Сергей Алекаандрович
(уч.степень) (должность) (подпись) (фамилия, имя, отчество)

Задание принял к исполнению: студент


(подпись) Аксёнов Дмитрий Игоревич
(фамилия, имя, отчество)

1. Введение

Компьютерное моделирование физических процессов является важным инструментом современного образования и науки. Особую ценность представляют интерактивные приложения, в которых пользователь может изменять параметры физической системы и сразу видеть, как это отражается на её поведении. Такой подход позволяет глубже понять взаимосвязь между формулами и реальной динамикой процесса.

В рамках данной работы рассматривается классический пример механических колебаний — пружинный маятник, представляющий собой груз массой m , закреплённый на пружине жёсткостью k . При смещении груза от положения равновесия возникает сила упругости $F = -kx$, возвращающая его обратно, что приводит к незатухающим гармоническим колебаниям (трение в модели не учитывается).

Ключевыми задачами при разработке приложения являются: построение математической модели, реализация её численного решения, проектирование графического интерфейса с элементами управления, организация анимированной визуализации пружины и построение графика смещения от времени в реальном времени.

2. Постановка задачи

Требуется разработать на языке Python с использованием библиотеки PySide6 (Qt for Python) интерактивное приложение, которое:

- Моделирует движение пружинного маятника путём численного интегрирования уравнения движения методом Эйлера.
- Визуализирует физическую систему в реальном времени: неподвижную опору, груз и пружину, отрисованную с помощью интерполяции кубическими сплайнами.
- Предоставляет пользователю элементы управления: **ползунки и числовые поля** для регулировки массы груза, жёсткости пружины и количества витков, а также кнопки **Старт**, **Пауза**, **Сброс** для управления симуляцией.
- Строит график зависимости координаты x от времени t по мере выполнения симуляции.

Расчётные формулы:

1. Дифференциальное уравнение движения (второй закон Ньютона):

$$m \cdot a = -k \cdot x$$

2. Преобразование к системе двух уравнений первого порядка:

$$dx/dt = v$$

$$dv/dt = a = -(k/m) \cdot x$$

3. Численное решение методом Эйлера с шагом Δt :

$$v(t + \Delta t) = v(t) + a(t) \cdot \Delta t$$

$$x(t + \Delta t) = x(t) + v(t) \cdot \Delta t$$

4. Аналитическое решение (для проверки) — гармоническая функция:

$$x(t) = A \cdot \cos(\omega t + \varphi), \text{ где } \omega = \sqrt{k/m}$$

5. Координаты опорных точек пружины (синусоида по витку):

$$y_i = y_{\text{подвеса}} + t_i \cdot L_{\text{пружины}}, x_i = x_{\text{подвеса}} + A \cdot \sin(2\pi \cdot n \cdot t_i)$$

где $L_{\text{пружины}}$ — текущая длина пружины в пикселях, A — амплитуда витка, n — количество витков, $t_i \in [0; 1]$ — нормированный параметр.

3. Инструкция к программной реализации

- **Запуск программы.** Запуск выполняется командой `python spring_pendulum.py`. При старте сразу открывается главное окно с настройками по умолчанию (масса 1 кг, жёсткость 10 Н/м, начальное смещение 10 см) и автоматически запущенной симуляцией.
- **Главное окно.** Окно разделено на две части. **Слева** размещён виджет анимации, отображающий опору, пружину и груз. **Справа** — панель управления, состоящая из трёх логических групп.
- **Параметры системы.** В этой группе расположены три пары «спинбокс + ползунок» для синхронного изменения **массы груза** (от 0.1 до 5.0 кг), **жёсткости пружины** (от 1.0 до 50.0 Н/м) и **количества**

витков пружины (от 3 до 30). Изменение этих параметров мгновенно отражается на анимации.

- **Управление.** Кнопки «Старт» и «Пауза» запускают и приостанавливают симуляцию без потери текущего состояния. Кнопка «Сброс» возвращает груз в начальное положение (смещение 0.10 м, скорость 0), очищает график и обнуляет время.
- **График $x(t)$.** В нижней части панели расположен график, построенный средствами QtCharts. По мере выполнения симуляции на нём отображается синусоидальная кривая смещения груза от времени, а ось времени автоматически расширяется.
- **Информационная строка.** Внизу панели расположена метка состояния, отображающая текущий режим работы программы: «Работа», «Пауза» или «Сброс».

4. Программный код и пояснения к нему

`spring_pendulum.py` — единый файл программы, содержащий точку входа и два основных класса: `SpringPendulumWidget` — виджет для отрисовки физической системы и `MainWindow` — главное окно приложения, отвечающее за логику симуляции, пользовательский интерфейс и связывание элементов управления с моделью.

4.1. Класс `SpringPendulumWidget` — виджет отрисовки маятника

Файл `spring_pendulum.py`, объявление класса и конструктор:

```
class SpringPendulumWidget(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setMinimumSize(500, 400)
        self.setStyleSheet("background-color: white;")
        self.pivot_x = 250      # X точки подвеса
        self.pivot_y = 50       # Y точки подвеса
        self.rest_length = 350  # Длина пружины в покое
        self.spring_width = 50  # Амплитуда витков
        self.num_coils = 12     # Количество витков
        self.mass = 1.0
        self.stiffness = 10.0
        self.x = 0.0
```

```
self.v = 0.0
self.px_per_meter = 50.0
```

В конструкторе:

- Устанавливается минимальный размер виджета 500×400 пикселей и белый фон через таблицу стилей.
- Задаются графические параметры: координаты точки подвеса (`pivot_x`, `pivot_y`), длина пружины в покое `rest_length`, амплитуда витков `spring_width` и их количество `num_coils`.
- Инициализируется физическое состояние: смещение `x`, скорость `v`, а также масштабный коэффициент `px_per_meter = 50`, переводящий метры в пиксели на экране.

Метод `set_physical_state()` и `get_mass_y_pixels()`:

```
def set_physical_state(self, x, v):
    self.x = x
    self.v = v

def get_mass_y_pixels(self):
    equilibrium_y = self.pivot_y + self.rest_length
    displacement_px = self.x * self.px_per_meter
    return equilibrium_y + displacement_px
```

Метод `set_physical_state()` вызывается из главного окна и обновляет внутреннее состояние виджета — текущие значения смещения и скорости. Метод `get_mass_y_pixels()` вычисляет Y-координату груза в пикселях: к Y-координате положения равновесия (точка подвеса плюс длина пружины в покое) прибавляется смещение, переведённое из метров в пиксели через множитель `px_per_meter`.

Метод `paintEvent()` — основная отрисовка сцены:

```
def paintEvent(self, event):
    painter = QPainter(self)
    painter.setRenderHint(QPainter.Antialiasing)
    # 1. Опора
    painter.setBrush(QBrush(QColor(100, 100, 100)))
    painter.drawRect(self.pivot_x - 50, self.pivot_y - 10, 100,
15)
    # 2. Пружина
```

```

self.draw_spring_interpolated(painter)
# 3. Груз
mass_y = self.get_mass_y_pixels()
painter.setBrush(QBrush(QColor(70, 130, 200)))
painter.setPen(QPen(Qt.black, 2))
painter.drawRect(self.pivot_x - 20, mass_y - 15, 40, 30)
painter.drawText(10, 20, f"Смещение: {self.x:.2f} м")

```

Метод `paintEvent()` переопределяет стандартный обработчик события перерисовки `QWidget` и автоматически вызывается каждый раз, когда система требует обновить содержимое виджета (в том числе после вызова `self.update()` из таймера). В нём:

- Создаётся объект `QPainter` и включается режим сглаживания (антиалиасинг) для качественной отрисовки кривых.
- Рисуются серая прямоугольная опора в верхней части сцены.
- Вызывается метод `draw_spring_interpolated()`, который строит пружину как гладкую кривую.
- Вычисляется текущая Y-координата груза, и в этой точке рисуется синий прямоугольник 40×30 пикселей с чёрной обводкой.
- В левом верхнем углу выводится текстовая подсказка с текущим значением смещения.

Метод `draw_spring_interpolated()` — отрисовка пружины через интерполяцию (часть 1, генерация опорных точек):

```

def draw_spring_interpolated(self, painter):
    mass_y = self.get_mass_y_pixels()
    spring_length_px = mass_y - self.pivot_y
    if spring_length_px <= 0:
        return
    num_points = self.num_coils * 8 # 8 точек на виток
    points = []
    for i in range(num_points + 1):
        t = i / num_points
        y = self.pivot_y + t * spring_length_px
        angle = 2 * math.pi * self.num_coils * t
        amplitude = self.spring_width * (1 - 0.2 * abs(2*t - 1))
        x = self.pivot_x + amplitude * math.sin(angle)
        points.append((x, y))

```

На этом этапе:

- Вычисляется текущая длина пружины в пикселях как разность между Y-координатой груза и точки подвеса. Если пружина сжата до нуля или отрицательной длины, отрисовка прерывается.
- Количество опорных точек выбрано как `num_coils * 8` — по 8 точек на каждый виток, что обеспечивает достаточную гладкость кривой.
- В цикле параметр `t` пробегает значения от 0 до 1 (доля длины пружины). Координата Y растёт линейно от точки подвеса к грузу, а X отклоняется от центральной оси по закону синуса с угловой частотой $2\pi \cdot n \cdot t$, где `n` — число витков. Это создаёт характерный синусоидальный профиль пружины.
- Амплитуда витков уменьшается к концам пружины через множитель $(1 - 0.2 \cdot |2t - 1|)$, что визуально сглаживает переход от пружины к опоре и грузу.

Метод `draw_spring_interpolated()` (часть 2, построение гладкого пути через Catmull-Rom сплайны):

```
path = QPainterPath()
path.moveTo(points[0][0], points[0][1])
for i in range(len(points) - 1):
    p0 = points[i]
    p1 = points[i + 1]
    if i == 0:
        path.lineTo(p1[0], p1[1])
    else:
        p_prev = points[i - 1]
        p_curr = p0
        p_next = p1
        p_next2 = points[i + 1] if i + 1 < len(points) else
p1
        tension = 0.5
        cp1_x = p_curr[0] + (p_next[0] - p_prev[0]) *
tension / 3
        cp1_y = p_curr[1] + (p_next[1] - p_prev[1]) *
tension / 3
        cp2_x = p_next[0] - (p_next2[0] - p_curr[0]) *
tension / 3
```



```

        cp2_y = p_next[1] - (p_next2[1] - p_curr[1]) *
tension / 3
        path.cubicTo(cp1_x, cp1_y, cp2_x, cp2_y, p_next[0],
p_next[1])
        pen = QPen(QColor(50, 50, 150), 3)
        painter.setPen(pen)
        painter.setBrush(Qt.NoBrush)
        painter.drawPath(path)

```

Здесь реализована интерполяция **сплайнами Catmull-Rom** через кубические кривые Безье. Для каждой пары соседних точек $p_{curr} \rightarrow p_{next}$ вычисляются две контрольные точки $cp1$ и $cp2$, направление которых определяется касательными в этих точках. Касательная в точке оценивается как разность её соседей слева и справа:

$$\text{касательная}(p_i) \approx (p_{i+1} - p_{i-1})/2$$

Параметр `tension = 0.5` определяет «натяжение» сплайна — насколько контрольные точки удалены от опорных. Полученные четыре точки (две опорных и две контрольных) передаются в метод `path.cubicTo()`, который добавляет к пути кубическую кривую Безье. В итоге получается гладкая линия, проходящая через все опорные точки, имитирующая витки реальной пружины. Пружина отрисовывается тёмно-синим пером толщиной 3 пикселя.

4.2. Класс `MainWindow` — главное окно приложения

Файл `spring_pendulum.py`, объявление класса и конструктор:

```

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Моделирование пружинного маятника")
        self.setGeometry(100, 100, 1000, 600)
        self.x = 0.10          # начальное смещение 10 см
        self.v = 0.0
        self.time = 0.0
        self.mass = 1.0
        self.stiffness = 10.0
        self.timer = QTimer()
        self.timer.timeout.connect(self.update_simulation)
        self.dt = 0.02
        self.is_running = True

```

```
self.timer.start(int(self.dt * 1000))
self.setup_ui()
```

В конструкторе:

- Устанавливается заголовок окна и его исходные размеры — 1000×600 пикселей.
- Инициализируются физические переменные: начальное смещение $x = 0.10$ м (10 см), начальная скорость $v = 0$ и время $time = 0$, а также параметры системы — масса 1 кг и жёсткость 10 Н/м.
- Создаётся таймер `QTimer` с шагом $dt = 0.02$ с (50 кадров в секунду). Его сигнал `timeout` связывается со слотом `update_simulation()`, который будет вызываться при каждом тике.
- Флаг `is_running = True` означает, что симуляция запускается автоматически при старте программы.
- Вызывается метод `setup_ui()`, который строит весь графический интерфейс.

Метод `setup_ui()` — построение интерфейса (фрагмент с компоновкой):

```
def setup_ui(self):
    central_widget = QWidget()
    self.setCentralWidget(central_widget)
    main_layout = QHBoxLayout(central_widget)
    # Левая часть: виджет отрисовки
    self.spring_widget = SpringPendulumWidget()
    main_layout.addWidget(self.spring_widget, 2)
    # Правая часть: панель управления
    control_panel = QWidget()
    control_layout = QVBoxLayout(control_panel)
    control_layout.setAlignment(Qt.AlignTop)
```

Метод `setup_ui()` создаёт центральный виджет и применяет к нему горизонтальный компоновщик `QHBoxLayout`. Слева размещается экземпляр `SpringPendulumWidget`, занимающий долю 2 от свободного пространства. Справа — вертикальная панель управления, прижатая к верху окна (`Qt.AlignTop`) и занимающая долю 1.

Фрагмент создания группы «Параметры системы» (ползунок массы):

```

params_group = QGroupBox("Параметры системы")
params_layout = QVBoxLayout()
mass_layout = QHBoxLayout()
mass_layout.addWidget(QLabel("Масса (кг):"))
self.mass_spin = QDoubleSpinBox()
self.mass_spin.setRange(0.1, 5.0)
self.mass_spin.setSingleStep(0.1)
self.mass_spin.setValue(self.mass)
self.mass_spin.valueChanged.connect(self.on_mass_changed)
mass_layout.addWidget(self.mass_spin)
self.mass_slider = QSlider(Qt.Horizontal)
self.mass_slider.setRange(1, 50)
self.mass_slider.setValue(int(self.mass * 10))
self.mass_slider.valueChanged.connect(self.on_mass_slider)
mass_layout.addWidget(self.mass_slider)

```

Для каждого изменяемого параметра создаётся пара виджетов:

- **QDoubleSpinBox** (или **QSpinBox** для целочисленного количества витков) — числовое поле с кнопками увеличения/уменьшения и заданным диапазоном и шагом.
- **QSlider** — горизонтальный ползунок для быстрого визуального изменения значения. Так как ползунок оперирует целыми числами, для дробных параметров используется коэффициент 10 (например, `setRange(1, 50)` при `value = int(mass * 10)`).
- Сигналы `valueChanged` обоих виджетов связываются со своими слотами, обеспечивающими двустороннюю синхронизацию (см. ниже). Аналогично создаются группы для жёсткости и числа витков.

Кнопки управления и группа графика:

```

self.start_btn = QPushButton("Старт")
self.start_btn.clicked.connect(self.on_start)
self.pause_btn = QPushButton("Пауза")
self.pause_btn.clicked.connect(self.on_pause)
self.reset_btn = QPushButton("Сброс")
self.reset_btn.clicked.connect(self.on_reset)

self.chart = QChart()
self.chart.setTitle("Зависимость смещения от времени")
self.series = QLineSeries()

```

```
self.series.setName("x(t)")
self.chart.addSeries(self.series)
self.chart.createDefaultAxes()
self.chart.axes()[0].setTitleText("Время (с)")
self.chart.axes()[1].setTitleText("Смещение (м)")
self.chart.axes()[1].setRange(-0.75, 0.75)
```

Кнопки «Старт», «Пауза» и «Сброс» создаются как обычные `QPushButton` и связываются с соответствующими слотами.

Для построения графика используются классы из модуля `QtCharts`. Создаётся объект `QChart` (сам график), а на него добавляется `QLineSeries` — серия точек, соединённых линией. Метод `createDefaultAxes()` автоматически создаёт оси, после чего им устанавливаются заголовки «Время (с)» и «Смещение (м)», а вертикальная ось ограничивается диапазоном $[-0.75; 0.75]$ м.

Слоты синхронизации ползунка и спинбокса:

```
def on_mass_slider(self, value):
    self.mass = value / 10.0
    self.mass_spin.blockSignals(True)
    self.mass_spin.setValue(self.mass)
    self.mass_spin.blockSignals(False)

def on_mass_changed(self, value):
    self.mass = value
    self.mass_slider.blockSignals(True)
    self.mass_slider.setValue(int(value * 10))
    self.mass_slider.blockSignals(False)
```

Эти два слота решают важную проблему **двусторонней синхронизации** ползунка и числового поля. Когда пользователь двигает ползунок, нужно обновить значение в спинбоксе и наоборот. Однако простое `setValue()` на одном виджете вызовет его сигнал `valueChanged`, тот вызовет слот, который снова сделает `setValue()` на первом виджете — возникнет **бесконечная рекурсия сигналов**. Для предотвращения этого перед программным изменением значения вызывается `blockSignals(True)`, а сразу после — `blockSignals(False)`. Аналогично устроены слоты для жёсткости и числа витков.

Слоты управления симуляцией:

```
def on_start(self):
    self.is_running = True
    self.info_label.setText("Состояние: Работа")

def on_pause(self):
    self.is_running = False
    self.info_label.setText("Состояние: Пауза")

def on_reset(self):
    self.is_running = False
    self.x = 0.10
    self.v = 0.0
    self.time = 0.0
    self.series.clear()
    self.time_data.clear()
    self.x_data.clear()
    self.series.append(0.0, self.x)
    self.chart.axes()[0].setMax(5)
    self.info_label.setText("Состояние: Сброс")
```

Логика слотов:

- `on_start()` устанавливает флаг `is_running = True`, и при следующем тике таймера симуляция возобновляется с того места, на котором была остановлена.
- `on_pause()` сбрасывает флаг, останавливая обновление физической модели без её сброса. Сам таймер при этом продолжает тикать, но метод `update_simulation()` при `is_running = False` сразу возвращает управление, не меняя состояние.
- `on_reset()` возвращает систему в начальное состояние: смещение 0.10 м , нулевая скорость, нулевое время. График очищается через `self.series.clear()`, а диапазон оси X восстанавливается до 5 секунд.

Метод `update_simulation()` — численное интегрирование уравнения движения:

```
def update_simulation(self):
    if not self.is_running:
```

```

        return
    # a = -(k/m) * x
    acceleration = - (self.stiffness / self.mass) * self.x
    self.v += acceleration * self.dt
    self.x += self.v * self.dt
    self.time += self.dt

    if len(self.time_data) < 20:
        self.series.append(self.time, self.x)
        if self.time > self.chart.axes()[0].max():
            self.chart.axes()[0].setMax(self.time + 1)

    self.spring_widget.set_physical_state(self.x, self.v)
    self.spring_widget.update()

```

Это **ключевой метод программы**, вызываемый каждые 20 мс таймером. Здесь реализован **метод Эйлера** для численного решения системы дифференциальных уравнений первого порядка. На каждом шаге:

- Сначала вычисляется ускорение по второму закону Ньютона: $a = -(k/m) \cdot x$ — сила упругости делённая на массу.
- Затем обновляется скорость: $v \leftarrow v + a \cdot \Delta t$.
- Затем обновляется координата: $x \leftarrow x + v \cdot \Delta t$. Используется именно новая скорость, что соответствует полуявному (симплектическому) методу Эйлера, который сохраняет энергию системы лучше классического.
- На график добавляется новая точка (**time**, **x**). Если текущее время превысило максимум оси, она автоматически расширяется через `setMax()`.
- Новое физическое состояние передаётся в виджет отрисовки через `set_physical_state()`, после чего вызывается `self.spring_widget.update()` — этот вызов планирует перерисовку, после которой Qt автоматически вызовет `paintEvent()` виджета.

Точка входа в программу:

```

if __name__ == "__main__":
    app = QApplication(sys.argv)

```

```
window = MainWindow()
window.show()
sys.exit(app.exec())
```

Стандартный шаблон Qt-приложения: создаётся объект `QApplication`, который управляет жизненным циклом программы, обработкой событий и циклом сообщений. Создаётся главное окно, отображается методом `show()`, и запускается цикл обработки событий через `app.exec()`, возвращаемое значение которого передаётся в `sys.exit()` как код завершения программы.

5. Демонстрация работы программы (скриншоты) с описанием

При запуске программы открывается главное окно, в котором сразу начинается симуляция: груз отклонён от положения равновесия на 10 см и совершает гармонические колебания.

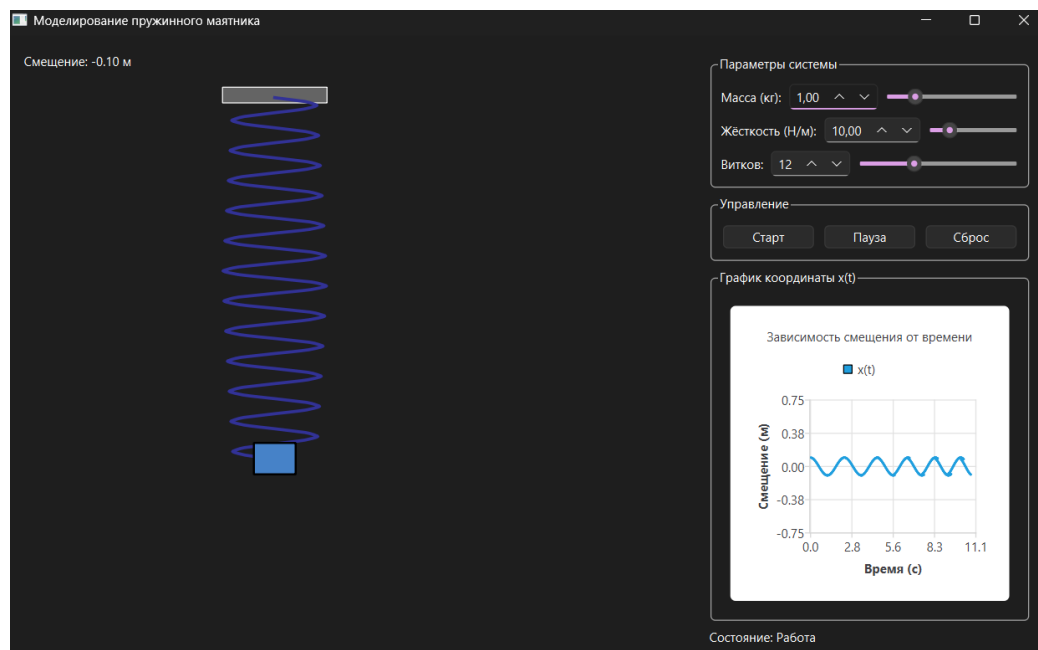


Рис. 5.1 — Главное окно приложения при запуске.

Слева видна сама физическая сцена: серая опора сверху, гладко отрисованная пружина и синий груз, который перемещается по вертикали. Справа расположена панель управления с тремя группами: *Параметры*

системы (масса, жёсткость, число витков), Управление (кнопки Старт/Пауза/Сброс) и График координаты $x(t)$ с синей синусоидой.

Пользователь может в любой момент изменить массу груза с помощью ползунка или числового поля. При увеличении массы период колебаний возрастает — груз качается заметно медленнее.



Рис. 5.2 — Изменение массы груза.

Аналогично, при увеличении жёсткости пружины (или уменьшении массы) частота колебаний возрастает, и груз качается быстрее. На графике это видно как уплотнение синусоиды.



Рис. 5.3 — Изменение жёсткости пружины.

Параметр «Витков» влияет только на визуализацию пружины — её гибкость и плотность намотки. На физику он не влияет, но позволяет добиться более реалистичного вида в зависимости от размеров пружины.

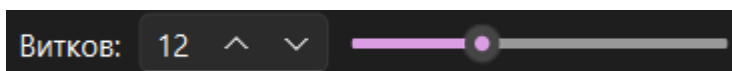


Рис. 5.4 — Изменение количества витков пружины.

Кнопка «Пауза» останавливает симуляцию без сброса состояния. Информационная строка внизу панели меняется на «Состояние: Пауза». Кнопка «Старт» продолжает движение с того же места.

Кнопка «Сброс» возвращает груз в начальное положение, обнуляет время и очищает график. Это удобно для повторного запуска эксперимента с другими параметрами.

6. Выводы

В ходе выполнения работы было разработано интерактивное приложение для моделирования и визуализации колебаний пружинного маятника. Приложение реализовано на языке Python с использованием

библиотеки PySide6 (Qt for Python) и модуля QtCharts для построения графиков.

В процессе разработки были изучены и применены ключевые принципы построения приложений с графическим интерфейсом на Qt: разделение приложения на функционально независимые классы (виджет отрисовки и главное окно), использование системы сигналов и слотов для связывания элементов управления с моделью, переопределение метода `paintEvent()` для пользовательской отрисовки, работа с таймером `QTimer` для организации анимации в реальном времени.

Я научился реализовывать численное интегрирование дифференциального уравнения движения методом Эйлера, рисовать сложные графические объекты через `QPainter` и `QPainterPath`, а также строить гладкие кривые с помощью интерполяции сплайнами Catmull-Rom через кубические кривые Безье. Отдельно был освоен приём двусторонней синхронизации ползунка и числового поля с защитой от рекурсии сигналов через `blockSignals()`.

Готовое приложение наглядно демонстрирует связь между параметрами физической системы (массой груза и жёсткостью пружины) и характеристиками её колебаний (периодом и частотой), что соответствует поставленной цели работы.